# Haute École d'Informatique

**AA 2025 - 2026 | Promotion K**

**UE : PROG3 - Implémentation d'API REST**

**Enseignant : M. Ryan A.**

# Exercice TD1 : Java & PostgreSQL (JDBC) - Filtre et pagination

**Timing : semaine du 1er décembre**

**Objectifs académiques :**

- Réussir à connecter une base de données PostgreSQL avec Java en utilisant JDBC.
- Comprendre comment doit être exploitée les données d'une base de données avec Java (ou tout autre langage orienté objet)
- Réussir à implémenter la pagination et les recherches multi-critères.

Considérons le schéma de la base de données suivant :

- Product (id: int, name: varchar, price: number, creation_datetime: timestamp)
- Product_category (id: int, name: varchar, #product_id: int)

1. Créez la base de données intitulée "product_management_db" ainsi qu'un nouvel utilisateur (hors ADMIN) qui va s'appeler "product_manager_user" avec un mot de passe "123456". Les privilèges doivent permettre à ce nouvel utilisateur de créer des tables sur cette nouvelle base de données et faire les CRUD sur les tables. Ajouter les scripts SQL utilisés dans un fichier "init_db.sql".

2. Créez les tables correspondantes au schéma fourni plus tôt et ajoutez les scripts SQL dans un nouveau fichier SQL intitulé "schema.sql".

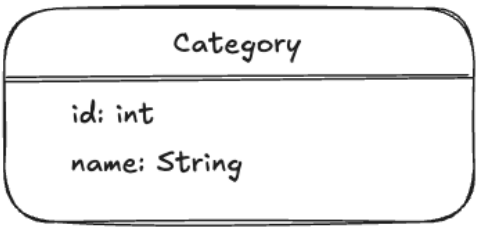
3. Insérez les données suivantes et ajouter les scripts SQL utilisés dans un nouveau fichier intitulé "data.sql" :
    a. Données de la table Product :

| id | name | price | creation_datetime |
|---|---|---|---|
| 1 | Laptop Dell XPS | 4500.00 | 2024-01-15 09:30:00 |
| 2 | iPhone 13 | 5200.00 | 2024-02-01 14:10:00 |
| 3 | Casque Sony WH1000 | 890.50 | 2024-02-10 16:45:00 |
| 4 | Clavier Logitech | 180.00 | 2024-03-05 11:20:00 |
| 5 | Ecran Samsung 27" | 1200.00 | 2024-03-18 08:00:00 |

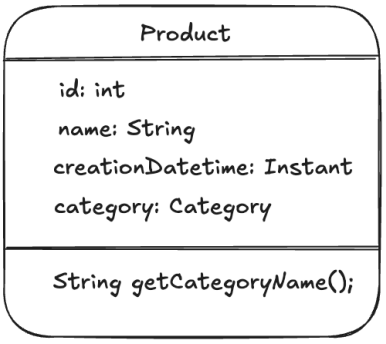
b. Données de la table Product_category :

| id | name | product_id |
|---|---|---|
| 1 | Informatique | 1 |
| 2 | Téléphonie | 2 |
| 3 | Audio | 3 |
| 4 | Accessoires | 4 |
| 5 | Informatique | 5 |
| 6 | Bureau | 5 |
| 7 | Mobile | 2 |

4. Créez les classes suivantes :
   a. Classe Category qui permet de spécifier une catégorie de produit :



b. Classe Product qui permet de spécifier un produit.




5. Créez la classe DBConnection qui permet de récupérer une Connection (représentant une connexion active entre l'apps Java et la base de données PostgreSQL) à travers une méthode : Connection getDBConnection();

En particulier, mettez en tant que variable **EN CLAIR** l'URL de la JDBC, le nom d'utilisateur et le mot de passe.

6. Créez une classe intitulée DataRetriever où vous allez récupérer les données Product et Category dans la base de données à travers Java en utilisant la classe DBConnection créée précédemment, et y ajouter les méthodes suivantes :

- **List<Category> getAllCategories()** : lire toutes les catégories enregistrées dans la base de données, les convertir et retourner une liste d'objets Category.

- **List<Product> getProductList (int page, int size)** : lire les produits enregistrés dans la base de données suivant une pagination, les convertir et retourner en liste d'objets Product. Par exemple, si page=1 et size=10, alors on va prendre les 10 premiers éléments; si page=2 et size=5, on va prendre les 6è au 10è éléments (donc 5 éléments du 2è page)

- **List<Product> getProductsByCriteria(String productName, String categoryName, Instant creationMin, instant creationMax)** : lire les produits enregistrés qui respectent les critères suivantes :

    - Si une valeur est fournie pour **productName**, filtrer la liste retournée avec la valeur contenue (opérateur ILIKE et non égale) notamment, ne retourner que les produits contenant un category contenant le nom de category fourni.. Si la valeur fournie est nulle, alors ne pas filtrer avec le nom du produit.

    - Si une valeur est fournie pour **categoryName**, filtrer la liste retournée avec la valeur contenue (opérateur ILIKE et non égale) notamment, ne retourner que les produits contenant un category contenant le nom de category fourni. Si la valeur fournie est nulle, alors ne pas filtrer avec le nom de la category.

    - Si une valeur **creationMin** est donnée, filtrer la liste retournée avec les produits qui ont été créés après la date et heure fournie si une valeur est

fournie. Si aucune valeur n'est fournie (nulle), alors ne pas filtrer avec une date et heure.

- Si une valeur **creationMax** est donnée, filtrer la liste retournée avec les produits qui ont été créés avant la date et heure fournie si une valeur est fournie. Si aucune valeur n'est fournie (nulle), alors ne pas filtrer avec une date et heure.

- **List<Product> getProductsByCriteria(String productName, String categoryName, Instant creationMin, instant creationMax, int page, int size) :** combine les filtres et paginations en même temps, il faut d'abord filtrer et ensuite paginer et non l'inverse.

7. Créez une classe Main, où vous allez invoquer toutes les méthodes de la classe DataRetriever pour les tester et affichez dans la console (avec la méthode statique **System.out.println()**) les résultats. En particulier, voici des données de tests en fonction des méthodes :

a) Pour **List<Category> getAllCategories() :** pas de paramètres alors ça va tout retourner.

b) Pour **List<Product> getProductList (int page, int size)** :

| page | size |
|---|---|
| 1 | 10 |
| 1 | 5 |
| 1 | 3 |
| 2 | 2 |

c) Pour **List<Product> getProductsByCriteria(String productName, String categoryName, Instant creationMin, instant creationMax) :**

| productName | categoryName | creationMin | creationMax |
|---|---|---|---|
| "Dell" | null | null | null |
| null | "info" | null | null |
| "iPhone" | "mobile" | null | null |
| null | null | 2024-02-01 | 2024-03-01 |
| "Samsung" | "bureau" | null | null |
| "Sony" | "informatique" | null | null |
| null | "audio" | 2024-01-01 | 2024-12-01 |
| null | null | null | null |

d) Pour **List<Product> getProductsByCriteria(String productName, String categoryName, Instant creationMin, instant creationMax, int page, int size)** :

| productName | categoryName | creationMin | creationMax | page | size |
|---|---|---|---|---|---|
| null | null | null | null | 1 | 10 |
| "Dell" | null | null | null | 1 | 5 |
| null | "informatique" | null | null | 1 | 10 |

**Anticipation pour la suite : Plutôt que de valider visuellement que tout marche sur une classe Main, documentez-vous sur JUnit, et écrivez des tests qui permettent de vérifier par programmation que vos fonctionnalités fonctionnent correctement;**

# Exercice TD2 : Java & PostgreSQL (JDBC) - Suite filtre, pagination et insertion de données

**Groupe K2 - 16 décembre 2025**
**Formulaire de dépôt : https://forms.gle/d2xfTimCCQicZkwx8**

Considérons le schéma de la base de données suivant permettant de représenter un système simplifié d'une gestion de restaurant :

- Ingredient (id: int, name: varchar, price: numeric, category : enum [VEGETABLE, ANIMAL, MARINE, DAIRY, OTHER], **#id_dish**: int)

- Dish (id: int, name: varchar, dish_type: enum [START, MAIN, DESSERT])

Notez que le type ENUM dans PostgreSQL est un type de données personnalisé permettant de définir une liste fixe et ordonnée de valeurs possibles pour une colonne, assurant ainsi l'intégrité des données.

Aussi, il faut savoir qu'il est possible qu'un ingrédient (Ingredient) ne soit pas associé à un plat (Dish).

1) Créez la nouvelle base de données intitulée "mini_dish_db", le nouvel utilisateur intitulé "mini_dish_db_manager" et associez les privilèges suffisants pour créer des tables et effectuer des opérations CRUD sur les tables. Ajoutez les scripts dans un fichier intitulé "db.sql".

2) Ajoutez les scripts permettant de créer les tables Ingredient et Dish, et ajoutez les scripts utilisés dans un fichier intitulé "schema.sql".

3) Insérez les données de tests suivants directement dans la base de données que vous allez utiliser plus tard, et écrivez les scripts dans un fichier intitulé "data.sql" :

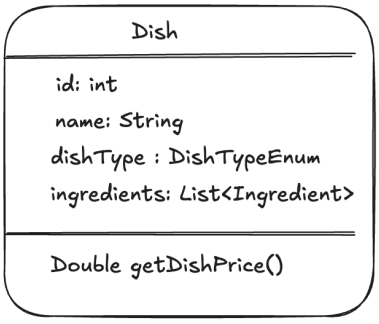
Dish :

| id | name | dish_type |
|---|---|---|
| 1 | Salade fraîche | START |
| 2 | Poulet grillé | MAIN |
| 3 | Riz aux légumes | MAIN |
| 4 | Gâteau au chocolat | DESSERT |
| 5 | Salade de fruits | DESSERT |

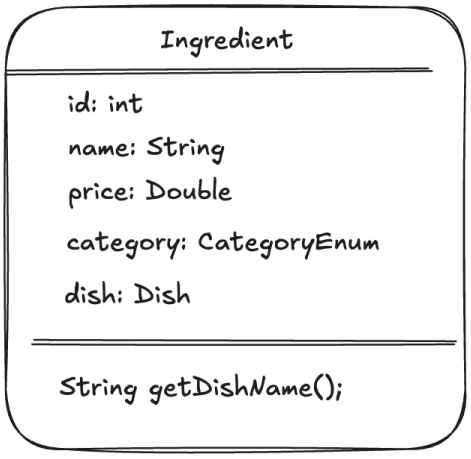
Ingredient :

| id | name | price | category | id_dish |
|---|---|---|---|---|
| 1 | Laitue | 800.00 | VEGETABLE | 1 |
| 2 | Tomate | 600.00 | VEGETABLE | 1 |
| 3 | Poulet | 4500.00 | ANIMAL | 2 |
| 4 | Chocolat | 3000.00 | OTHER | 4 |
| 5 | Beurre | 2500.00 | DAIRY | 4 |

4) Créez les classe suivantes :
   a) La classe Dish qui décrit un plat, ayant un identifiant, un nom et une liste des ingrédients qui le compose, n'oubliez pas de créer également la classe DishTypeEnum :



b) La classe Ingredient qui décrit un ingrédient, ayant un identifiant unique, un nom, un prix, une catégorie et appartenant à un plat, n'oubliez pas de créer également la classe Enum CategoryEnum :




5) Créez une classe DBConnection qui permet d'établir la connexion entre votre application Java et PostgreSQL, à travers une méthode : Connection getDBConnection(); En particulier, stockez dans des variables d'environnement la valeur des variables nécessaires pour établir la connexion, notamment JDBC_URL, USERNAME et PASSWORD.

   Pour rappel, la méthode en Java "System.getEnv("**nom_variable**"), permet de récupérer une variable d'environnement om "**nom_variable**" est le nom donné et pour le configurer, vous pouvez voir directement dans IntelliJ.

   Vous pouvez également utiliser la libraire `dotenv` si vous voulez faciliter la gestion des variables d'environnements.

6) Créez une classe intitulée DataRetriever où vous allez récupérer les données Dish et Ingredient depuis la base de données à travers Java en utilisant la classe DBConnection créée précédemment, et y implémenter les méthodes suivantes en veillant à utiliser PreparedStatement au lieu de Statement lorsqu'il le faut :

   a) **Dish findDishById(Integer id)** permettant de récupérer un plat en fournissant son identifiant unique (`id`), mais également tous les ingrédients qui le composent.

   b) **List<Ingredient> findIngredients(int page, int size)** permettant de récupérer la liste des ingrédients à travers une pagination.

   c) **List<Ingredient> createIngredients(List<Ingredient> newIngredients)** permettant de créer de nouveaux ingrédients. Si un ingrédient existe déjà dans la nouvelle liste fournie, alors il faut lever une uncheked exception (RuntimeException) et annuler toute l'opération (notamment si d'autres

ingrédients ont été déjà sauvegardés dans la base de données, alors les enlever). Pour gérer cette annulation, il faut se renseigner sur le **principe de l'atomicité :** soit tout (les ingredients) est enregistré, soit rien n'est enregistré.

d) **Dish saveDish(Dish dishToSave)** permettant de sauvegarder un plat dans la base de données, dans le cas où le plat n'existe pas encore alors il faut l'insérer, et dans le cas où il existe, alors il faut le mettre à jour. En particulier, cette méthode permet d'associer ou de dissocier un ingrédient (ou une liste d' ingrédients) d'un plat.

e) **List<Dish> findDishsByIngredientName(String IngredientName)** permettant de récupérer la liste des plats qui possède un nom des ingrédients contenant la valeur du paramètre `IngredientName` fourni à travers la méthode.

f) **List<Ingredient> findIngredientsByCriteria(String ingredientName, CategoryEnum category, String dishName, int page, int size)** permettant de récupérer la liste des ingrédients suivant les critères : nom d' ingrédient, sa catégorie **ET** le nom du plat auquel il est associé, et tout cela de manière paginée. En particulier, si aucune valeur n'est fournie pour être filtrée, alors ignorez la condition.

7) Tests à faire pour vous assurer que tout est conforme à ce qui est attendu :
   a) **Dish findDishById(Integer id);** id = 1; Résultat attendu : Salade Fraîche avec 2 ingrédients (Laitue et Tomate)

   b) **Dish findDishById(Integer id);** id = 999; Résultat attendu : RuntimeException

   c) **List<Ingredient> findIngredients(int page, int size);** page=2; size=2; Résultat attendu : Ingrédients Poulet, Chocolat

   d) **List<Ingredient> findIngredients(int page, int size);** page=3; size=5; Résultat attendu : Liste vide

   e) **List<Dish> findDishsByIngredientName(String IngredientName);** IngredientName="eur"; Résultat attendu : Plat - Gâteau au choclat

   f) **List<Ingredient> findIngredientsByCriteria(...);** ingredientName=null, category=VEGETABLE, dishName=null, page=1, size=10; Résultat attendu = Laitue, Tomate

   g) **List<Ingredient> findIngredientsByCriteria(...);** ingredientName="cho", category=null, dishName="Sal", page=1, size=10; Résultat attendu = Liste vide

h) **List<Ingredient> findIngredientsByCriteria(...);** ingredientName="cho", category=null, dishName="gâteau", page=1, size=10; Résultat attendu = Chocolat

i) **List<Ingredient> createIngredients(...);** [Ingredient (name=1 category=DAIRY, price=1200.0), Ingredient (name=Oignon, cagetory=VEGETABLE, price=500.0)] Résultat attendu=Liste des ingrédients Fromage et Oignons créés et retournés

j) **List<Ingredient> createIngredient(...)** [Ingredient (name=Carotte category=VEGETABLE, price=2000.0), Ingredient (name=Laitue, cagetory=VEGETABLE, price=2000.0)] Résultat attendu = Exception levée car laitue est un ingrédient déjà existant

k) **Dish saveDish(...) :** Dish(name=Soupe de légumes, dishType=START, ingrédients=[Oignon]); Résultat attendu = Dish (soupe de légumes) créé avec succès contenant l'ingrédient Oignon

l) **Dish saveDish(...)** : Dish(id=1, name=Salade fraîche, dishType=START, ingrédients=[Oignon, Laitue, Tomate, Fromage]); Résultat attendu = Dish avec les ingrédients mis à jour (ajout de l'oignon et du fromage)

m) **Dish saveDish(...)** : Dish(id=1, name=Salade de fromage, dishType=START, ingrédients=[Fromage]); Résultat attendu = Dish avec les ingrédients mis à jour (suppression des ingrédients oignons, laitue et tomate; il ne reste que fromage)

# Exercice TD3 : Java & PostgreSQL (JDBC) - Normalisation de données et gestion de relation ManyToMany

**Date d'émission : Jeudi 15 janvier 2026 | Correction : Lundi 20 janvier 2026**

Nous pouvons remarquer qu'une entité "ingrédient" appartient à une entité "plat" ce qui veut dire qu'un ingrédient avec un même nom peut être inséré plusieurs fois dans la base de données mais avec différents identifiants et associés à différents plats.

Par exemple, reconsidérons l'exemple du plat "Salade fraîche" composé des ingrédients : "Laitue" et "Tomate". Voici comment sont enregistrés les données dans la base de données pour correspondre à ce besoin :

Dans la table Dish :

| id | name | dish_type |
|---|---|---|
| 1 | Salade fraîche | START |

Dans la table Ingredient :

| id | name | price | category | id_dish |
|---|---|---|---|---|
| 1 | Laitue | 800.00 | VEGETABLE | 1 |
| 2 | Tomate | 600.00 | VEGETABLE | 1 |

Imaginons maintenant que nous voulons ajouter un nouveau plat "Salade de tomates", dont l'ingrédient est également "Tomate". Nous ne pouvons dissocier l'ingrédient (id=2, name="Tomate", id_dish=1) pour l'associer au nouveau plat mais nous allons devoir devoir créer une nouvel ingrédient pour correspondre au "Tomate", donc avec les même propriétés, mais seule valeur de id_dish change en celle correspondant à l'identifiant du nouveau plat "Salade de tomates".

Autrement dit, voici comment sont enregistrés les données dans la base de données :

Dans la table Dish :

| id | name | dish_type |
|---|---|---|
| 6 | Salade de tomates | STARTER |

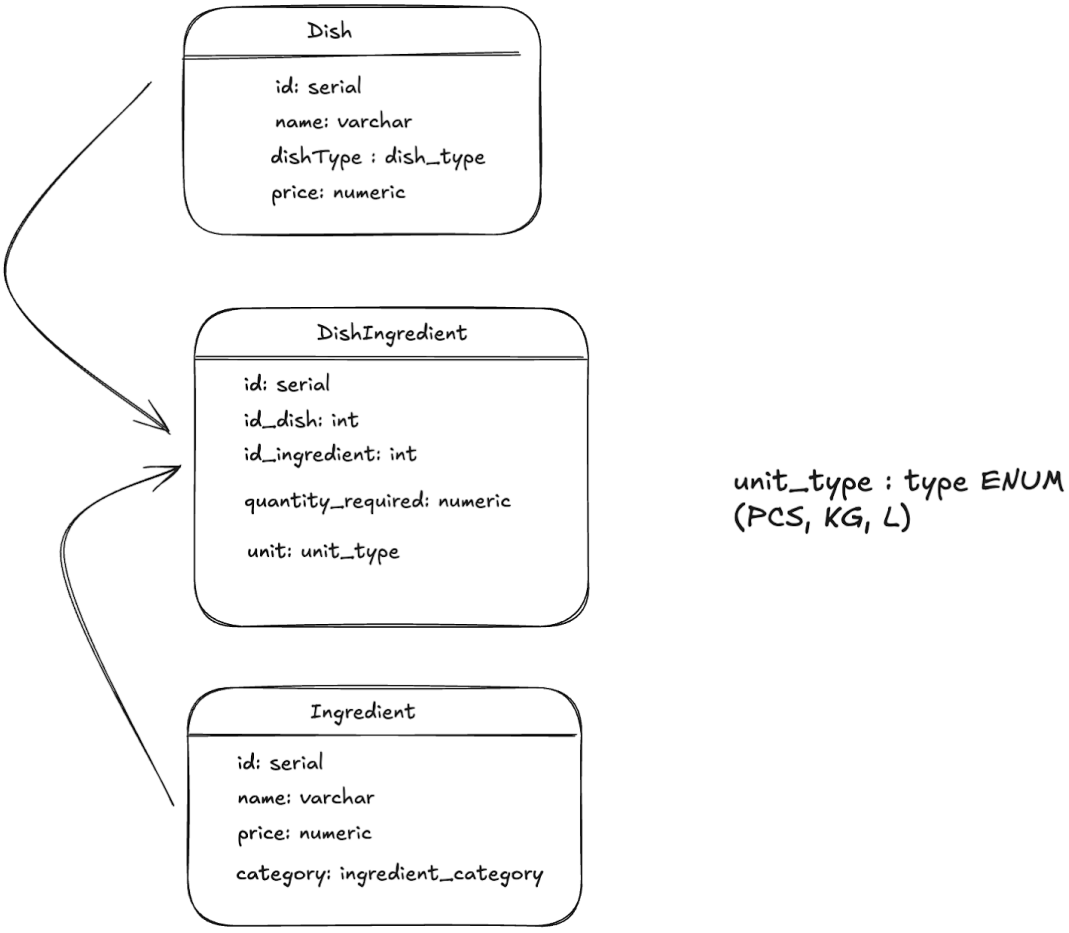
Dans la table Ingredient :

| id | name | price | category | id_dish |
|---|---|---|---|---|
| 6 | Tomate | 600.0 | VEGETABLE | 6 |

Hormis les remarques déjà effectuées plus tôt, un problème réside dans la mise à jour des propriétés de cet ingrédient, notamment si on veut mettre à jour le prix des tomates alors il faut mettre à jour toutes les lignes d'ingrédients avec un nom "Tomate", où il suffit qu'on ait écrit "tomate" au lieu de "Tomate" ou encore "Tomates" (au pluriel) pour que ça ne marche pas (en plus des problème de performance).

Pour gérer ceci plus efficacement, il faut donc que nous ajustons les relations des tables au sein de notre base de données où nous allons devoir gérer une relation ManyToMany et appliquer ce que l'on appelle la **NORMALISATION DE DONNÉES** (https://www.datacamp.com/fr/tutorial/normalization-in-sql)

Pour cela, il faut que vous ajouter une nouvelle table de jointure entre Ingredient et Dish que nous allons appeler "DishIngredient" et qui va avoir la structure suivante :




Dorénavant, un ingrédient est enregistré comme un seul élément que vous pouvez directement associer à plusieurs plats en même temps à travers la table de jointure DishIngredient.

Remarquez que dans cette nouvelle table, nous spécifions également la quantité requise d'un ingrédient pour composer un plat, ainsi que l'unité utilisée pour la quantité. Voici quelques exemples pour comprendre cela :

- Une salade fraîche est composée de laitue et de tomate, mais il faut être plus précis sur cette composition. On peut par exemple dire qu'il faut une pièce de laitue et 0,25 KG de tomates, en particulier car de façon générale, lorsqu'on achète ces ingrédients, on achète la laitue par pièce et les tomates par unité de kilogramme.

- Un autre exemple, pour le plat poulet grillé, enrichissons un peu les ingrédients : il faut 0,5 KG de poulet et 0,15 L d'huile, encore une fois car lorsque nous achetons du poulet, on achète par unité en KG et de l'huile en unité en litre (L).

1. Effectuer les modifications nécessaires pour normaliser les données auprès de la base de données Postgresql et écrire les scripts correspondant à ces modifications dans un nouveau fichier "**new_schema.sql**". En particulier, il faut que vous ajoutiez, si ce n'est pas fait un prix de vente à un plat, qui est optionnel (peut être null).

2. Insérez les données suivantes
   a. Dans la table DishIngredient :

| id | id_dish | id_ingredient | quantity_required | unit |
|---|---|---|---|---|
| 1 | 1 | 1 | 0.20 | KG |
| 2 | 1 | 2 | 0.15 | KG |
| 3 | 2 | 3 | 1.00 | KG |
| 4 | 4 | 4 | 0.30 | KG |
| 5 | 4 | 5 | 0.20 | KG |

   b. Dans la table Dish (pour la mise à jour des prix) :

| id | name | dish_type | selling_price |
|---|---|---|---|
| 1 | Salade fraîche | START | 3500.00 |
| 2 | Poulet grillé | MAIN | 12000.00 |
| 3 | Riz aux légumes | MAIN | NULL |
| 4 | Gâteau au chocolat | DESSERT | 8000.00 |
| 5 | Salade de fruits | DESSERT | NULL |

3. Effectuez les modifications nécessaires dans DataRetriever pour maintenir les méthodes existantes (en y ajoutant de nouvelles méthodes car il y a une nouvelle entité à sauvegarder) notamment la récupération de données (tout ce qui est méthode qui commence par **find** et la sauvegarde de données tout ce qui est méthode qui commence par **save**).

4. Mettre à jour la méthode **Double getDishCost()** en y intégrant les quantités requises dans le calcul. Pour le moment, nous n'allons pas considérer la conversion des unités.

5. Ajouter la méthode permettant de calculer la marge brute de chaque plat en la méthode précédente et en ajoutant, si ce n'est pas encore le cas, un prix de vente aux plats. Il faut savoir que c'est un attribut pouvant être nul, auquel cas une exception pourrait être levée durant le calcul si c'est ce qui se produit.

Voici les valeurs attendues pour les données de tests actuels :

- Pour la méthode getDishCost() :

| Plat | Coût attendu |
|---|---|
| Salade fraîche | **250.00** |
| Poulet grillé | **4500.00** |
| Riz aux légumes | **0.00** |
| Gâteau au chocolat | **1400.00** |
| Salade de fruits | **0.00** |

- Pour la méthode getGrossMargin() :

| Plat | Marge attendue |
|---|---|
| Salade fraîche | **3250.00** |
| Poulet grillé | **7500.00** |
| Riz aux légumes | ❌ **Exception (prix NULL)** |
| Gâteau au chocolat | **6600.00** |
| Salade de fruits | ❌ **Exception (prix NULL)** |

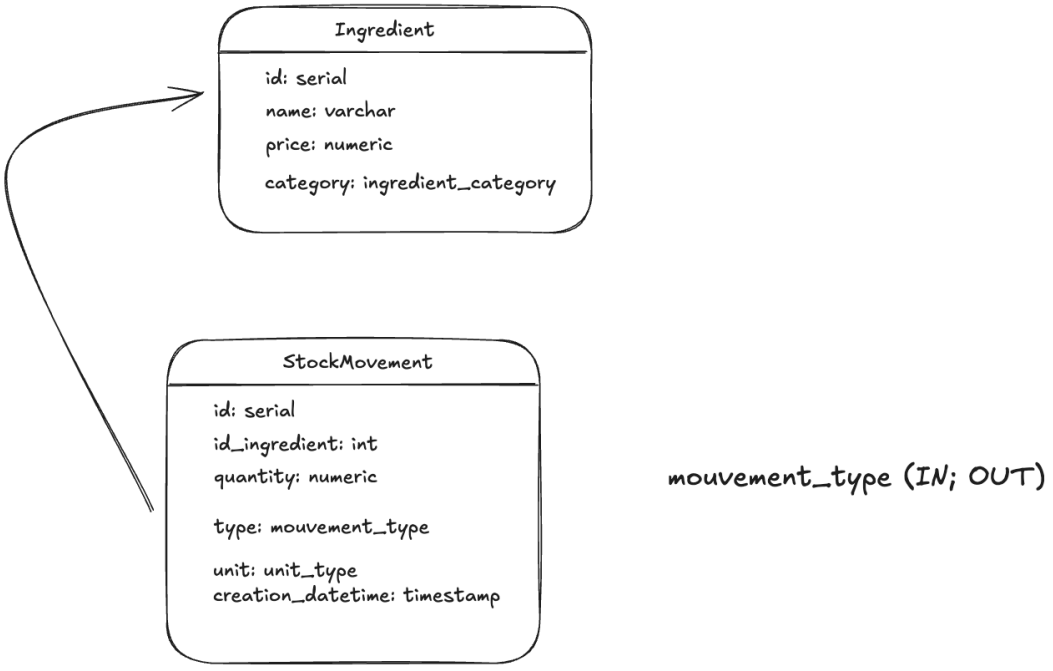
# Exercice TD4 : Java & PostgreSQL (JDBC) - Gestion de stocks

Dans un contexte réel, il est essentiel pour un restaurant de pouvoir gérer les stocks de ses ingrédients. En particulier, il faut acheter les ingrédients pour augmenter le niveau de stock et à chaque vente réalisée, le stock diminue en fonction des quantités nécessaires et du nombre de plats vendus.

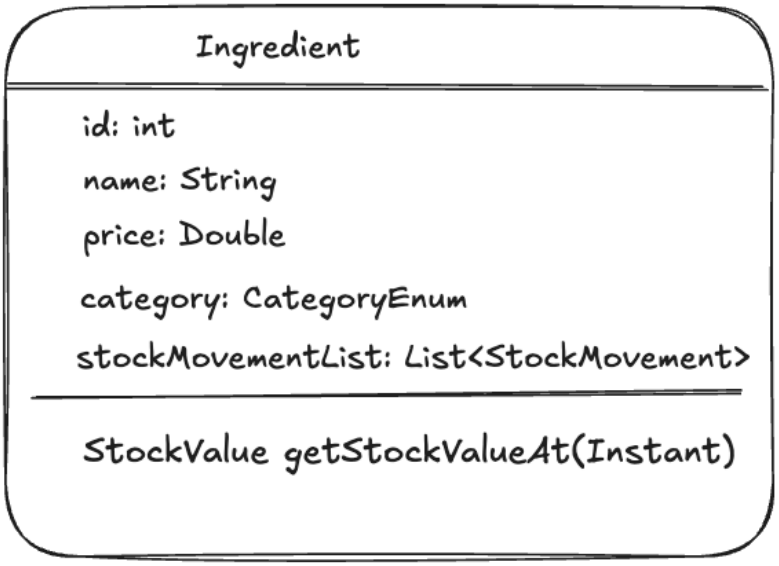
Dans cette étape, nous allons essentiellement considérer le mouvement des stocks pour le moment. Un mouvement de stock ou `StockMovement` est caractérisé par :

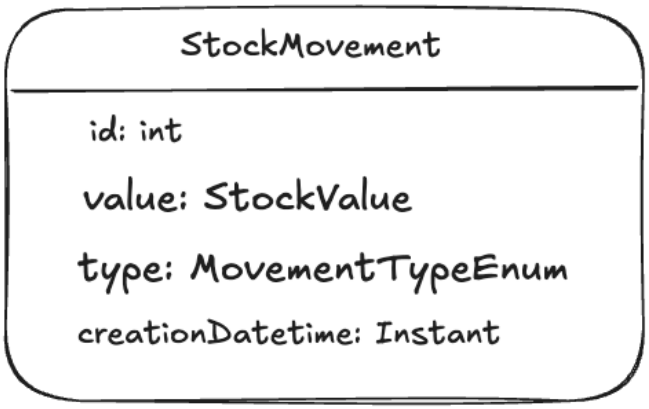
- Un identifiant unique;
- L'ingrédient correspondant au mouvement;
- Le type du mouvement;
- La quantité du mouvement ainsi que l'unité correspondant;
- La date et heure du mouvement.

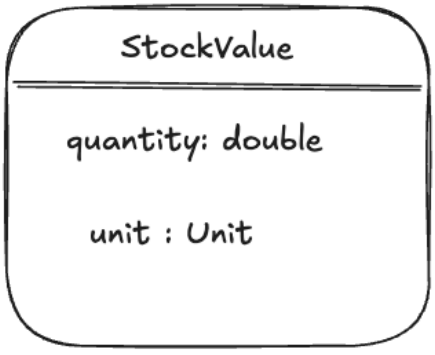
Voici les changements correspondants à l'ajout des mouvements des stocks au niveau de la base de données :



Au niveau applicatif, voici la nouvelle structure des classes attendues au niveau de Java :




1) Effectuez les modifications nécessaires au niveau de la base de données pour pouvoir gérer les stocks et ajoutez les données suivantes.

📊 **StockMovement**

| id | id_ingredient | quantity | type | unit | creation_datetime |
|---|---|---|---|---|---|
| 1 | 1 | 5.0 | IN | KG | 2024-01-05 08:00 |
| 2 | 1 | 0.2 | OUT | KG | 2024-01-06 12:00 |
| 3 | 2 | 4.0 | IN | KG | 2024-01-05 08:00 |
| 4 | 2 | 0.15 | OUT | KG | 2024-01-06 12:00 |
| 5 | 3 | 10.0 | IN | KG | 2024-01-04 09:00 |
| 6 | 3 | 1.0 | OUT | KG | 2024-01-06 13:00 |
| 7 | 4 | 3.0 | IN | KG | 2024-01-05 10:00 |
| 8 | 4 | 0.3 | OUT | KG | 2024-01-06 14:00 |
| 9 | 5 | 2.5 | IN | KG | 2024-01-05 10:00 |
| 10 | 5 | 0.2 | OUT | KG | 2024-01-06 14:00 |

2) Effectuez les modifications nécessaires au niveau de la classe DataRetriever en suivant les consignes intermédiaires suivantes. Notez que nous ne considérons pas encore la conversion des unités, d'où toutes les unités sont en KG.

   a) Créer la méthode `Ingredient saveIngredient(Ingredient toSave);`

   b) Lorsqu'on veut ajouter un mouvement à un Ingredient, il faut lui attribuer à travers l'attribut `stockMovementList` et lors du sauvegarde (effectuée par la méthode `saveIngredient`). En particulier, il faut utiliser l'identifiant unique du mouvement de stock pour différencier la création et la mise à jour : si un identifiant est existant et qu'on essaie d'effectuer une insertion, alors il faut en rien faire (`on conflict do nothing` durant l'insertion) sinon, on ajoute le nouveau mouvement. Aussi, il n'est pas possible de supprimer ou d'enlever les mouvements de stock.

3) Utilisez les données de tests suivants pour les calculs du niveau de stock de chaque ingrédient.

   Voici le niveau de stock initial à considérer :

| Ingrédient | Stock |
|---|---|
| Laitue | 5.0 |
| Tomate | 4.0 |
| Poulet | 10.0 |
| Chocolat | 3.0 |
| Beurre | 2.5 |

Vérifiez les résultats retournés par la méthode `getStockValueAt(Instant t)` pour tous les ingrédients en utilisant `t = 2024-01-06 12:00`, en ajoutant les mouvements de stocks :

| id | id_ingredient | quantity | type | unit | creation_datetime |
|---|---|---|---|---|---|
| 6 | 1 | 0.2 | OUT | KG | 2024-01-06 12:00 |
| 7 | 2 | 0.15 | OUT | KG | 2024-01-06 12:00 |
| 8 | 3 | 1.0 | OUT | KG | 2024-01-06 12:00 |
| 9 | 4 | 0.3 | OUT | KG | 2024-01-06 12:00 |
| 10 | 5 | 0.2 | OUT | KG | 2024-01-06 12:00 |

Le niveau de stock de chaque ingrédient (donc Stock::getQuantity) est :

| Ingrédient | Calcul | Stock |
|---|---|---|
| Laitue | 5.0 – 0.2 | **4.8** |
| Tomate | 4.0 – 0.15 | **3.85** |
| Poulet | 10.0 – 1.0 | **9.0** |
| Chocolat | 3.0 – 0.3 | **2.7** |
| Beurre | 2.5 – 0.2 | **2.3** |